

Signal and Image Processing

Assignment 5 - Features and Scale-Space Theory

Group 8

Simone Skovgaard: wnf255

Noah Junge: qxk266

Marcus Friis-Hansen: lns611

February 2026

1 Inspecting Spectrograms

1.1

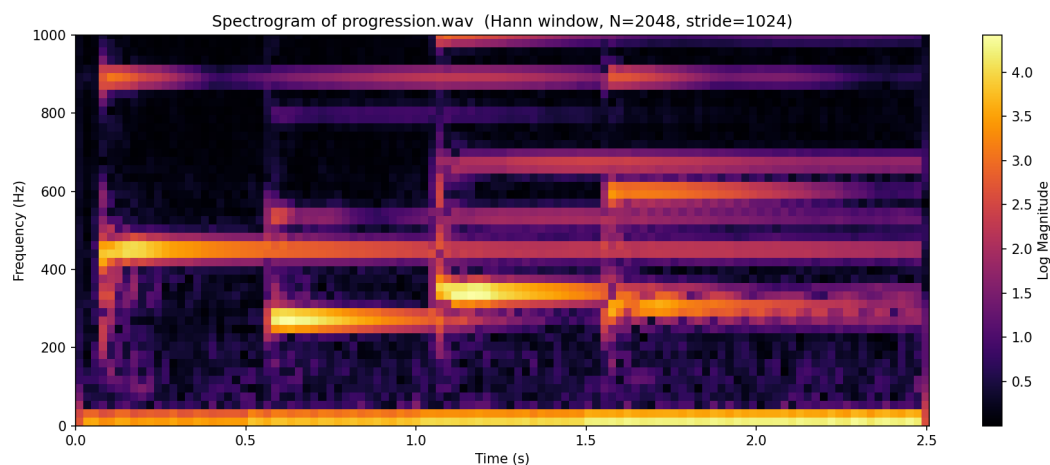


Figure 1: Spectrogram of progression.wav

```
def compute_spectrogram(audio, sample_rate, window_size):  
  
    stride = window_size // 2  
    f, t, Zxx = scipy_stft(  
        audio,  
        fs=sample_rate,  
        window="hann",  
        nperseg=window_size,  
        noverlap=window_size - stride,  
    )  
    magnitude = np.abs(Zxx)  
    return magnitude, t, f
```

We calculated the stride based on the window size and then use `scipy_stft` to compute the short-time Fourier transform using Hann windows. `scipy_stft` returns frequency f and time values t , and the complex Fourier coefficients, which we take the absolute value of find the magnitude. Frequency is the y-axis, time is the x -axis, and the magnitude is the intensity of each frequency band in the finished spectrogram.

By finding the lowest frequency with the most energy in the spectrogram in Figure 1, we can identify the notes being played by comparing them to Table 1 from the assignment description. The notes being played are A - C - E - D. The window size used to produce the spectrogram was 2048 with a stride of $\frac{2048}{2} = 1024$. We found that it resulted in a good enough resolution to allow us to discern the notes. The window size affects the frequency resolution and time resolution of the spectrogram. A small window gives a lower frequency resolution and a higher time resolution, and a big window gives a higher frequency resolution but a lower time resolution.

2 Inverse filtering

2.1

```
def embed_kernel(kernel, shape):
    """
    Embed kernel in array of given shape with kernel center at (0,0).
    Required for correct circular convolution via FFT.
    """
    M, N = shape
    kh, kw = kernel.shape
    k_embed = np.zeros((M, N))
    k_embed[:kh, :kw] = kernel
    k_embed = np.roll(k_embed, -(kh // 2), axis=0)
    k_embed = np.roll(k_embed, -(kw // 2), axis=1)
    return k_embed

def apply_lsi(image, kernel, noise):
    """Apply LSI degradation:  $G = H * F + N$  via circular convolution (FFT)"""
    k_embed = embed_kernel(kernel, image.shape)
    degraded = np.real(iff2(fft2(image) * fft2(k_embed)))
    return degraded + noise
```

We implement $g = f * h + n$ by computing the FFT of the image and the embedded kernel then multiply in the frequency domain. Then we perform inverse transforming to obtain the blurred image and apply the noise realization. The LSI degradation model demonstrates two independent degradation mechanisms. There is blur, controlled by the PSF parameter σ_k and noise, controlled by σ_n . Increasing σ_k removes edges and fine texture, while increasing σ_n adds disturbance across all frequencies.

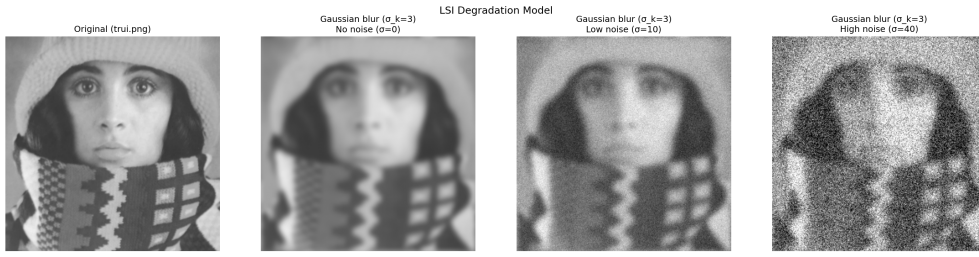


Figure 2: LSI Degradation model on trui.png $\sigma_k = 3$

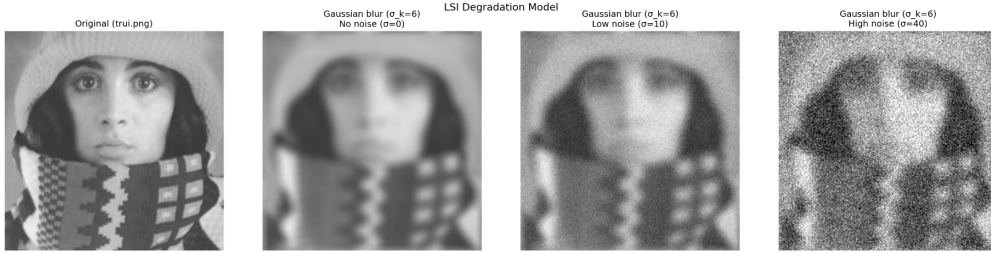


Figure 3: LSI Degradation model on trui.png with $\sigma_k = 6$

2.2

```
def direct_inverse_filter(degraded, kernel, epsilon=1e-3):
    """
    Direct inverse filter:  $\hat{F} = G/H$ .
    Uses epsilon threshold to avoid division by near-zero  $H$  values.
    """
    G = fft2(degraded)
    H = fft2(embed_kernel(kernel, degraded.shape))
    H_safe = np.where(np.abs(H) > epsilon, H, epsilon)
    return np.real(ifft2(G / H_safe))
```

We have implemented direct inverse filtering in the frequency domain based on the LSI degradation model, and then taking the Fourier transform. By assuming we know the PSF function, we estimate the original image by dividing in the frequency domain. The restored image can then be obtained by inverse FFT.

When no noise is present ($\sigma_n = 0$) direct inverse filtering is able to largely recover the sharpness of the original image, since the degradation is purely due to blur and can be mathematically inverted. In the presence of noise the method becomes highly unstable. Even moderate noise levels lead to restorations dominated by high-frequency noise and for extreme noise levels the result becomes unusable. From the results obtained, it is evident that direct inverse filtering works only in nearly noise-free conditions and is not robust to realistic noise levels.

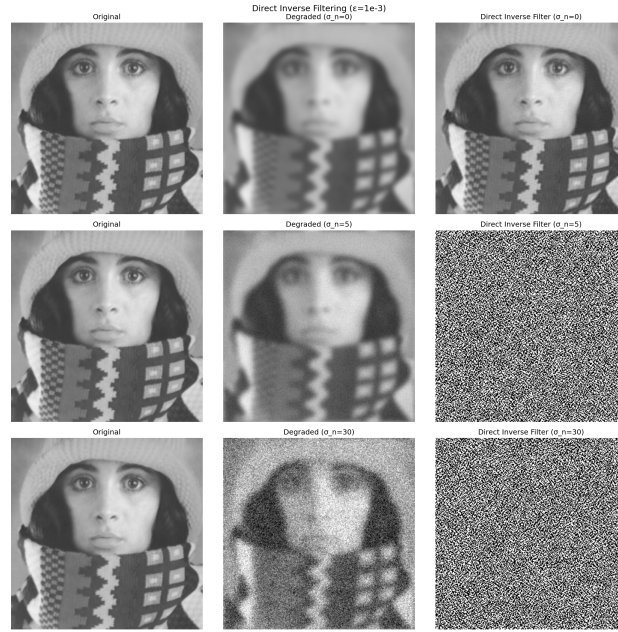


Figure 4: Direct Inverse filtering applied to LSI degraded trui.png

2.3

```
def wiener_filter(degraded, kernel, K):
    """
    Wiener filter (Gonzales & Woods):
    W = H* / (|H|^2 + K)
    K approximates the noise to signal power spectral density ratio.
    """
    G = fft2(degraded)
    H = fft2(embed_kernel(kernel, degraded.shape))
    W = np.conj(H) / (np.abs(H)**2 + K)
    return np.real(iff2(W * G))
```

We implement Wiener filtering in the frequency domain by taking the FFT of the degraded image and the known PSF. Wiener filtering then applies a stabilized inverse, it inverts frequencies where the PSF is reliable and suppresses frequencies where inversion would amplify noise. The parameter K controls this. Larger K means stronger noise suppression but less sharpening. In the final step, we transform back to the spatial domain using the inverse FFT.

Compared to direct inverse filtering, the Wiener filter is more robust when noise is present. Small K (0.001) gives more sharpening but tends to amplify noise, especially at high noise levels. Larger K (0.1) produces cleaner restorations by damping noisy high-frequency components, but leaves more blur and reduces fine details. Wiener filtering gives a balance between deblurring and denoising, but still remains usable at extreme noise levels.



Figure 5: Wiener Filter applied to LSI degraded trui.png

3 Fixed scale feature detectors

3.1

```
img = load_image("hand.tiff")
img_norm = img / img.max() # skimage canny expects float in [0,1]

configs = [
    (1.0, 0.05, 0.15, "sigma=1.0_low=0.05_high=0.15\n(default-like)"),
    (3.0, 0.05, 0.15, "sigma=3.0_low=0.05_high=0.15\n(smooth, fewer edges)"),
    (1.0, 0.02, 0.06, "sigma=1.0_low=0.02_high=0.06\n(sensitive, more edges)"),
]

for i, (sigma, low, high, label) in enumerate(configs):
    edges = canny(img_norm, sigma=sigma, low_threshold=low,
                  high_threshold=high)
```

Increasing sigma suppresses fine detail and produces smoother, more continuous edges, and decreasing it preserves small structures but may introduce noise. Lower threshold values increase sensitivity and detect more edges, including weak and noisy gradients. Higher thresholds produce cleaner but sparser edge maps. From the results it is clear that the parameters should be chosen depending on whether detail preservation or noise suppression is more important.



Figure 6: Different settings of parameters on the Canny function

3.2

```
img = load_image("modelhouses.png")
img_norm = img / img.max()

# method='k' variations
configs_k = [
    (1.0, 0.05, "method=k\nsigma=1,\nk=0.05"),
    (3.0, 0.05, "method=k\nsigma=3,\nk=0.05"),
    (2.0, 0.15, "method=k\nsigma=2,\nk=0.15"),
]

# method='eps' variations (fixed sigma=2)
configs_eps = [
    (2.0, 0.01, "method=eps\nsigma=2,\neps=0.01"),
    (2.0, 0.1, "method=eps\nsigma=2,\neps=0.1"),
]

for i, (sigma, k, label) in enumerate(configs_k):
    response = corner_harris(img_norm, method="k", sigma=sigma, k=
        k)

for i, (sigma, eps, label) in enumerate(configs_eps):
    response = corner_harris(img_norm, method="eps", sigma=sigma,
        eps=eps)
```

Increasing sigma increases spatial smoothing, which suppresses small-scale features, produces more stable but fewer corner responses and focuses on larger structures. For larger k, there is stricter corner detection and fewer responses. For smaller k the detector more sensitive and there are more detected features. When using method='eps', changing the parameter eps from 0.1 to 0.01, it is difficult to see a visual difference. The choice of parameters determines the trade-off between sensitivity to small details and robustness to noise.

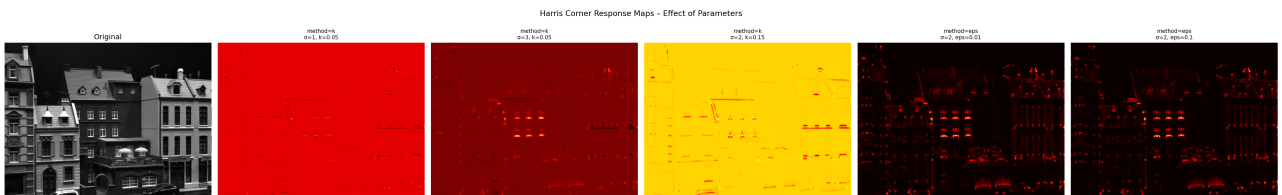


Figure 7: Computed Harris corner response image with different parameter settings

3.3

```
def find_harris_corners(image, num_peaks=250, sigma=2.0, k=0.05,
    min_distance=5):
    """Find Harris corners as local maxima of the corner response map.
    """
    response = corner_harris(image, method="k", sigma=sigma, k=k)
    coords = corner_peaks(response, num_peaks=num_peaks, min_distance=
        min_distance)
    return coords, response

img = load_image("modelhouses.png")
img_norm = img / img.max()

sigma, k = 2.0, 0.05
coords, _ = find_harris_corners(img_norm, num_peaks=250, sigma=
    sigma, k=k)

ax.imshow(img, cmap="gray")
ax.plot(coords[:, 1], coords[:, 0], "r.", markersize=4, label=f"
    Corners (n={len(coords)})
```



Figure 8: 250 strongest Harris Corners with method=k, $\sigma = 2.0$, $k = 0.05$, minimum distance = 5

4 Scale-space blob detector

4.1

$$G(x, y, \sigma) * G(x, y, \tau) = G(x, y, \sqrt{\sigma^2 + \tau^2}) \quad (1)$$

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (2)$$

$$B(x, y) = G(x, y, \sigma) \quad (3)$$

$$I(x, y, \tau) = B(x, y) * G(x, y, \tau) \quad (4)$$

The convolution of two Gaussian functions is also a Gaussian, which gives eq. (1). A Gaussian is defined as shown in eq. (2). A function made from this analytical expression is shown in eq. (3).

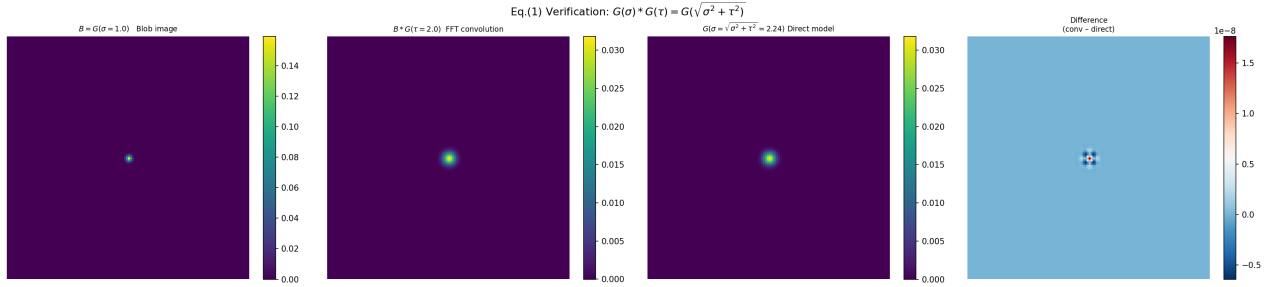


Figure 9: Gaussian blobs for problem 4.1 sampled to a 128×128 size image

The first image in figure 9 above shows a 2-dimensional Gaussian with scale $\sigma = 1$, the second image shows this Gaussian convolved with a Gaussian with scale $\tau = 2$. The third image shows a model blob with scale $\sqrt{\sigma^2 + \tau^2}$. When you compare these two images, they look visually identical, which confirms eq. (1). The fourth image visualizes the difference between the second and third images. The fourth image shows that there is a very small patterned difference. The scale bar shows that the difference at most is $1.5 \cdot 10^{-8}$, which is an insignificant difference for most purposes. The error occurs because both images are discrete approximations of a continuous Gaussian sampled on a finite 128×128 grid, but the convolution operation performed on image 2 introduces new approximation error, which isn't present in image 3.

```
sigma, tau = 1.0, 2.0
sigma_combined = np.sqrt(sigma**2 + tau**2)
B = gaussian_2d(X, Y, sigma)
G_tau = gaussian_2d(X, Y, tau)
G_comb = gaussian_2d(X, Y, sigma_combined)
result_conv = np.real(iff2(fft2(B) * fft2(G_tau)))
diff = result_conv - G_comb
```

Above, we show the most important lines of code for this problem. They include the definition of $\sigma = 1$, $\tau = 2$, the combined $\sqrt{\sigma^2 + \tau^2}$, as well as the construction of different Gaussians. Although not shown above, we have sampled X and Y used in the Gaussian prior. Next line we perform the convolution and compute the difference between image 2 and 3.

4.2

I will prove that $G(x, y, \sigma) * G(x, y, \tau) = G(x, y, \sqrt{\sigma^2 + \tau^2})$ using the Fourier Transform and the convolution theorem. Please note that there is a notational inconsistency with the Gaussian in the spatial vs frequency domain in this assignment, when compared to the course slides. I will therefore use $G()$ for the Gaussian in the spatial domain, and $\hat{G}()$ for the Gaussian in the frequency domain. We perform the Fourier transform on the Gaussian function. This is shown on slide 25 of FourierII.pdf.

$$\hat{G}(u) = \int_{-\infty}^{\infty} G(x) e^{-i2\pi ux} dx = \exp\left(\frac{-\pi^2 u^2}{a}\right)$$

We convert this to the sigma notation used in the assignment description. We can do this by comparing the exponential function used in the slide to the standard Gaussian formula:

$$G(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}, \quad G(x) = \sqrt{\frac{a}{\pi}} e^{-ax^2}, \quad a = \frac{1}{2\sigma^2}$$

This leads to

$$\hat{G}(u) = \exp\left(\frac{-\pi^2 u^2}{\frac{1}{2\sigma^2}}\right) = e^{-\pi^2 u^2 2\sigma^2}$$

Extending the Gaussian to 2-dimensions in the frequency domain is simple, as its Fourier transform is separable $\hat{F}(u, v) = \hat{G}(u)\hat{H}(v)$.

$$\hat{G}(u, v) = e^{-\pi^2 u^2 2\sigma^2} \cdot e^{-\pi^2 v^2 2\sigma^2} = e^{-2\pi^2 (u^2 + v^2) \sigma^2}$$

I will define each Gaussian for clarity

$$\hat{G}(u, v, \sigma) = e^{-2\pi^2 (u^2 + v^2) \sigma^2}, \quad \hat{G}(u, v, \tau) = e^{-2\pi^2 (u^2 + v^2) \tau^2}, \quad \hat{G}(u, v, \sigma) \cdot \hat{G}(u, v, \tau) = \hat{G}(u, v, \theta)$$

We now multiply the two 2-dimensional Gaussians together to perform the convolution. As per the convolution theorem, A convolution in the spatial representation leads to a product in the frequency representation.

$$\hat{G}(u, v, \sigma) \cdot \hat{G}(u, v, \tau) = e^{-2\pi^2 (u^2 + v^2) \sigma^2} \cdot e^{-2\pi^2 (u^2 + v^2) \tau^2} = e^{-2\pi^2 (u^2 + v^2) (\sigma^2 + \tau^2)}$$

From here it follows that

$$\hat{G}(u, v, \theta) = e^{-2\pi^2 (u^2 + v^2) \theta^2} = e^{-2\pi^2 (u^2 + v^2) (\sigma^2 + \tau^2)}$$

which makes $\theta^2 = \sigma^2 + \tau^2 \Rightarrow \theta = \sqrt{\sigma^2 + \tau^2}$. And therefore

$$\hat{G}(u, v, \theta) = \hat{G}(u, v, \sqrt{\sigma^2 + \tau^2}) = e^{-2\pi^2 (u^2 + v^2) (\sigma^2 + \tau^2)}$$

We now use the inverse Fourier transform to transform the Gaussians back to the spatial domain, which concludes the proof

$$G(x, y, \sigma) * G(x, y, \tau) = G(x, y, \sqrt{\sigma^2 + \tau^2})$$

4.3

Consider the 2-dimensional scale normalized partial derivatives of order $m + n$ at scale τ

$$l_{x^m y^n}(x, y, \tau) = \tau^{\gamma(m+n)} \frac{\partial^{m+n} I(x, y, \tau)}{\partial x^m \partial y^n}$$

where $\gamma \in \mathbb{R}$ is a parameter of the scale normalization and $I(x, y, \tau)$ is the scale space of an image. Now consider the normalized Laplacian of the blob image defined in eq. (3), where $\gamma = 1$

$$H(x, y, \tau) = I_{xx}(x, y, \tau) + I_{yy}(x, y, \tau)$$

4.3.1

I will write the closed-form expression for $H(x, y, \tau)$, so I first need to find $I_{xx}(x, y, \tau)$ and $I_{yy}(x, y, \tau)$. We can use equations(1),(3) and (4) to expand.

$$I(x, y, \tau) = B(x, y) * G(x, y, \tau) = G(x, y, \sigma) * G(x, y, \tau) = G(x, y, \sqrt{\sigma^2 + \tau^2})$$

to simplify we will define the scale as $\theta = \sqrt{\sigma^2 + \tau^2}$

$$I(x, y, \tau) = G(x, y, \sqrt{\sigma^2 + \tau^2}) = G(x, y, \theta) = \frac{1}{2\pi\theta^2} e^{-\frac{x^2+y^2}{2\theta^2}}$$

We now define the scale normalized derivatives for $I_{xx}(x, y, \tau)$ and $I_{yy}(x, y, \tau)$:

$$I_{xx}(x, y, \tau) = \tau^1 \frac{\partial^2 I(x, y, \theta)}{\partial x^2}, \quad I_{yy}(x, y, \tau) = \tau^1 \frac{\partial^2 I(x, y, \theta)}{\partial y^2}$$

I find the first partial derivative

$$\frac{\partial I}{\partial x} = \frac{1}{2\pi\theta^2} \cdot \left(-\frac{x}{\theta^2}\right) e^{-\frac{x^2+y^2}{2\theta^2}}$$

Then the second partial derivative

$$\frac{\partial^2 I}{\partial x^2} = \left(-\frac{1}{\theta^2}\right) \frac{1}{2\pi\theta^2} e^{-\frac{x^2+y^2}{2\theta^2}} + \left(\frac{x^2}{\theta^4}\right) \frac{1}{2\pi\theta^2} e^{-\frac{x^2+y^2}{2\theta^2}} = \left(\frac{x^2}{\theta^4} - \frac{1}{\theta^2}\right) \frac{1}{2\pi\theta^2} e^{-\frac{x^2+y^2}{2\theta^2}}$$

Due to the Gaussian being a symmetric function, the second partial derivative for y is identical, with y^2 instead of x^2 .

$$\frac{\partial^2 I}{\partial y^2} = \left(\frac{y^2}{\theta^4} - \frac{1}{\theta^2}\right) \frac{1}{2\pi\theta^2} e^{-\frac{x^2+y^2}{2\theta^2}}$$

we then add them together

$$\frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2} = \left(\frac{x^2}{\theta^4} - \frac{1}{\theta^2}\right) \frac{1}{2\pi\theta^2} e^{-\frac{x^2+y^2}{2\theta^2}} + \left(\frac{y^2}{\theta^4} - \frac{1}{\theta^2}\right) \frac{1}{2\pi\theta^2} e^{-\frac{x^2+y^2}{2\theta^2}} = \left(\frac{x^2+y^2}{\theta^4} - \frac{2}{\theta^2}\right) \frac{1}{2\pi\theta^2} e^{-\frac{x^2+y^2}{2\theta^2}}$$

We also need to multiply by τ^1 for scaling. Below i show the closed form, where $\theta = \sqrt{\sigma^2 + \tau^2}$

$$H(x, y, \tau) = \tau \left(\frac{x^2+y^2}{\theta^4} - \frac{2}{\theta^2}\right) \frac{1}{2\pi\theta^2} e^{-\frac{x^2+y^2}{2\theta^2}}$$

4.3.2

I need to analytically derive the scale(s) for τ , for the point $(x, y) = (0, 0)$ meaning $H(0, 0, \tau)$. I plug $(x, y) = (0, 0)$ into the closed-form expression I derived in the problem above.

$$H(0, 0, \tau) = \tau \left(\frac{0}{\theta^4} - \frac{2}{\theta^2} \right) \frac{1}{2\pi\theta^2} e^{-\frac{0}{2\theta^2}} = \frac{-2\tau}{2\pi\theta^4} e^0 = \frac{-\tau}{\pi\theta^4} = \frac{-\tau}{\pi\sqrt{\sigma^2 + \tau^2}^4} = \frac{-\tau}{\pi(\sigma^2 + \tau^2)^2}$$

To find the extrema with respect to τ , we need to find the points where the derivative equals zero. I will first derive it in Maple.

$$\frac{d}{d\tau} \frac{-\tau}{\pi(\sigma^2 + \tau^2)^2} = -\frac{1}{\pi(\sigma^2 + \tau^2)^2} + \frac{4\tau^2}{\pi(\sigma^2 + \tau^2)^3}$$

We then solve for τ when it's equal to zero. I multiply both sides by the denominator in the second term: $\pi(\sigma^2 + \tau^2)^3$

$$-\frac{1}{\pi(\sigma^2 + \tau^2)^2} + \frac{4\tau^2}{\pi(\sigma^2 + \tau^2)^3} = 0 \Rightarrow -(\sigma^2 + \tau^2) + 4\tau^2 = 0 \Rightarrow -\sigma^2 - \tau^2 + 4\tau^2 = 0$$

i isolate τ and σ by adding σ^2 to both sides:

$$3\tau^2 = \sigma^2 \Rightarrow \tau = \frac{\sigma}{\sqrt{3}}$$

So there is an extremal point whenever $\tau = \frac{\sigma}{\sqrt{3}}$

We can characterize these extrema(s) by analyzing the behavior of $H(0, 0, \tau)$ for values of τ around $\tau = \frac{\sigma}{\sqrt{3}}$. I choose values that are simple to compute and compare:

$$H(0, 0, \tau = 0) = 0, \quad H(0, 0, \tau = \frac{\sigma}{\sqrt{3}}) < 0, \quad H(0, 0, \tau = \infty) = 0$$

This means that each extrema when $\tau = \frac{\sigma}{\sqrt{3}}$ is a minimum since the function declines when $\tau < \frac{\sigma}{\sqrt{3}}$ and grows when $\tau > \frac{\sigma}{\sqrt{3}}$

4.3.3

We confirm the result of problem 4.3.2 by plotting $H(0, 0, \tau)$ as a function of τ .

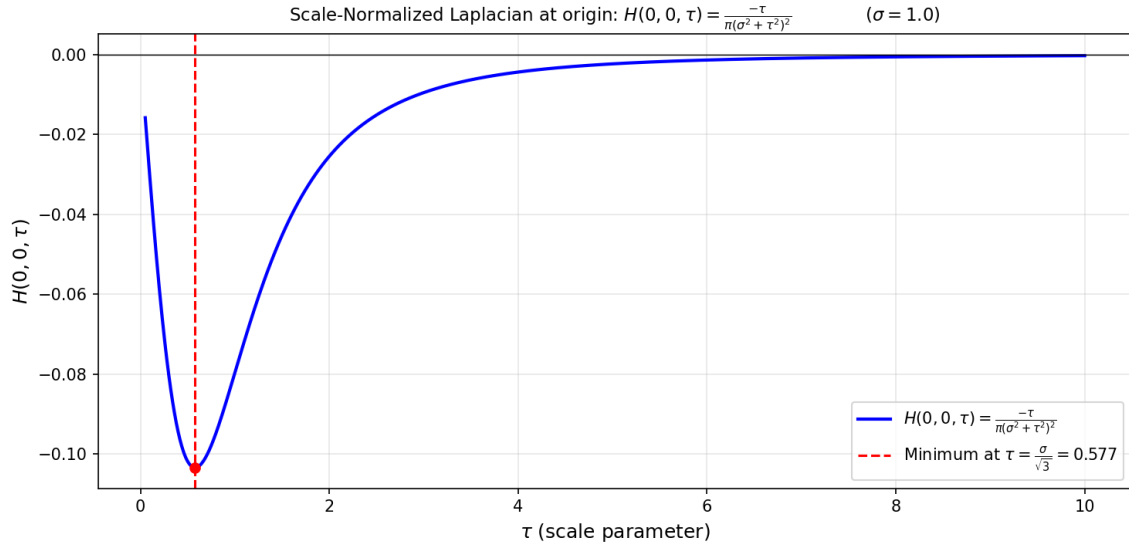


Figure 10: Plot of $H(0, 0, \tau)$ as a function of τ for an arbitrary value of $\sigma = 1$

In the plot in figure 10 we can see that $H(0, 0, \tau)$ declines until $\tau = \frac{\sigma}{\sqrt{3}} = 0.577$ where it reaches a minimum. It then grows closer to 0 as τ grows. This confirms the result of problem 4.3.2. that $\tau = \frac{\sigma}{\sqrt{3}}$ is a minima.

4.4

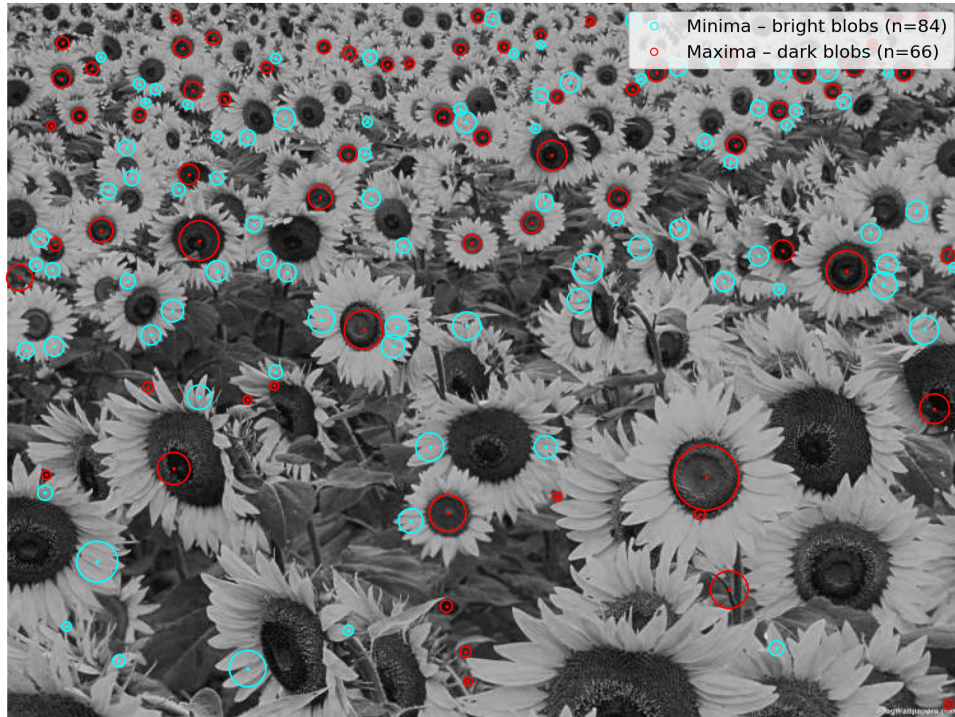


Figure 11: Blob detection on sunflower.tiff

Above in figure 11 is an illustration of our results. The image shows the 150 maxima (red) or minima (cyan) blobs with the largest value of $H(x, y, \tau)$. Maxima represent dark blobs, which in this case are the dark centers of the sunflowers. Minima represent bright blobs, which in this case are the bright flower petals of the sunflowers in the image. The dark blobs are maxima due to the pixel intensity increasing from the center to the edges of the blobs. Similarly, the bright blobs are minima due to the pixel intensity decreasing from the center to the edges.

Below, we showcase the most relevant lines of code. We essentially create a scale space of the image using a Laplacian of Gaussian, and then use `peak_local_max` to find local maxima and minima in the image.

```
tau_vals = np.logspace(np.log10(TAU_MIN), np.log10(TAU_MAX), N_SCALES)
H_stack = np.zeros((N_SCALES, img.shape[0], img.shape[1]))
for i, tau in enumerate(tau_vals):
    H_stack[i] = tau * gaussian_laplace(img_norm, sigma=tau)

coords_max = peak_local_max(H_stack, min_distance=MIN_DISTANCE,
                             threshold_abs=0.0)
coords_min = peak_local_max(-H_stack, min_distance=MIN_DISTANCE,
                             threshold_abs=0.0)
```